

WEEK1 SKILL

Session1:

- 1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.**

Task -1: Create Project Structure as follows:

Note: For creating this structure use mkdir and touch commands.

```
manikanta@Hemu:~/OSSP$ tree
```

```
.
├── LICENSE
├── Makefile
├── README.md
├── assets
│   └── architecture.png
├── bin
│   └── my_shell
├── docs
│   ├── design.md
│   └── report.pdf
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── scripts
│   └── run.sh
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── fork_exec.c
│   ├── fork_exec.c.save
│   ├── main.c
│   ├── parser.c
│   ├── shell.c
│   ├── task2
│   ├── task2.c
│   └── task2.c.save
└── tests
    ├── test_executor.c
    └── test_parser.c
```

```
9 directories, 24 files
```

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;

    pid = fork();    // Create a new process

    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        // Child Process
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
    else {
        // Parent Process
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }

    return 0;
}
```

```
manikanta@Hemu:~/OSSP/task2$ gcc task2.c -o task2
```

```
manikanta@Hemu:~/OSSP/task2$ ./task2
```

```
This is the Parent Process.
```

```
Parent PID = 860
```

```
Child PID = 861
```

```
This is the Child Process.
```

```
Child PID = 861
```

Report: I created the necessary folders and files, initialized a Git repository, and verified the project structure using the tree command. Thus, I successfully completed the task and understood how to organize a project in Linux.

To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing.

Task-1: Using a manual command understand the fork() and exec() system calls

Fork()

NAME

fork - create a child process

LIBRARY

Standard C library (`libc`, `-lc`)

SYNOPSIS

```
#include <unistd.h>
```

```
pid_t fork(void);
```

DESCRIPTION

`fork()` creates a new process by duplicating the calling process. The new process is referred to as the child process. The calling process is referred to as the parent process.

The child process and the parent process run in separate memory spaces. At the time of `fork()` both memory spaces have the same content. Memory writes, file mappings (`mmap(2)`), and unmappings (`munmap(2)`) performed by one of the processes do not affect the other.

The child process is an exact duplicate of the parent process except for the following points:

- The child has its own unique process ID, and this PID does not match the ID of any existing process group (`setpgid(2)`) or session.
- The child's parent process ID is the same as the parent's process ID.
- The child does not inherit its parent's memory locks (`mlock(2)`, `mlockall(2)`).
- Process resource utilizations (`getrusage(2)`) and CPU time counters (`times(2)`) are reset to zero in the child.
- The child's set of pending signals is initially empty (`sigpend-`

Manual page fork(2) line 1 (press h for help or q to quit)

Man exec:

```

exec(3)                                Library Functions Manual                                exec(3)

NAME
    execl, execlp, execl, execv, execvp, execvpe - execute a file

LIBRARY
    Standard C library (libc, -lc)

SYNOPSIS
    #include <unistd.h>

    extern char **environ;

    int execl(const char *path, const char *arg, ...
                /*, (char *) NULL */);
    int execlp(const char *file, const char *arg, ...
                /*, (char *) NULL */);
    int execl(const char *path, const char *arg, ...
                /*, (char *) NULL, char *const envp[] */);
    int execv(const char *path, char *const argv[]);
    int execvp(const char *file, char *const argv[]);
    int execvpe(const char *file, char *const argv[], char *const envp[]);

    Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

    execvpe():
        _GNU_SOURCE

DESCRIPTION
    The exec() family of functions replaces the current process image
    with a new process image. The functions described in this manual
    page are layered on top of execve(2). (See the manual page for ex-
    ecve(2) for further details about the replacement of the current
    process image.)

    The initial argument for these functions is the name of a file that
    is to be executed.

    The functions can be grouped based on the letters following the
    "exec" prefix.

```

Report: I used the manual commands `man fork` and `man exec` to understand the `fork()` and `exec()` system calls. I read the manual pages and learned that `fork()` creates a new child process, while `exec()` replaces the current process with a new program. Thus, I successfully understood the purpose and working of both system calls.

Task-2: Write a C program to create a new process and accepting the user commands using fork() and exec() system calls.

```
GNU nano 8.7.1 task2.c
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;

    pid = fork();    // Create a new process

    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        // Child Process
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
    else {
        // Parent Process
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }

    return 0;
}
```

```
manikanta@Hemu:~/OSSP/task2$ gcc task2.c -o task2
manikanta@Hemu:~/OSSP/task2$ ./task2
This is the Parent Process.
Parent PID = 1156
Child PID = 1157
This is the Child Process.
Child PID = 1157
```

Report: I wrote and executed a C program using the fork() system call to create a child process. I compiled and ran the program successfully and observed the creation of both the parent and child processes along with their Process IDs (PIDs). Thus, I understood the working of the fork() system call and the parent-child process relationship successfully.

SESSION 2:

1. To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

GNU nano 8.7.1

```
#include <stdio.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>
#define BUFFER_SIZE 100
int main() {
    char buffer[BUFFER_SIZE];
    int index;
    struct termios oldt, newt;
    // Get current terminal settings
    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;
    // Disable canonical mode
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);
    while (1) {
        index = 0;
        printf("myshell> ");
        fflush(stdout);
        while (1) {
            char ch = getchar();
            // Enter key
            if (ch == '\n') {
                buffer[index] = '\0';
                printf("\n");
                // Exit condition
                if (strcmp(buffer, "exit") == 0) {
                    printf("Exiting shell...\n");
                    tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
                    return 0;
                }
                printf("You entered: %s\n", buffer);
                break;
            }
            // Backspace
            else if (ch == 127 || ch == '\b') {
                if (index > 0) {
```

```
manikanta@Hemu:~/shell_project$ gcc shell.c -o shell
manikanta@Hemu:~/shell_project$ ./shell
myshell> hello
You entered: hello
myshell> test123
You entered: test123
myshell> exit
Exiting shell...
manikanta@Hemu:~/shell_project$ |
```

Report: I compiled and ran the shell program and tested it with the inputs hello, test123, and exit. I observed that the shell correctly displayed the entered commands and exited when the exit command was given. Thus, I successfully implemented and tested the main loop, user input handling, and exit condition of the shell program.

2. To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define BUFFER_SIZE 100

int main() {
    char command[BUFFER_SIZE];

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        // Read user input
        if (fgets(command, BUFFER_SIZE, stdin) == NULL) {
            break;
        }

        // Remove newline
        command[strcspn(command, "\n")] = '\0';

        // Exit condition
        if (strcmp(command, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        // Ignore empty input
        if (strlen(command) == 0) {
            continue;
        }

        // Create child process
        pid_t pid = fork();

```

Report: I wrote and executed a C program using fgets() to read user input, handled the newline character, implemented the exit condition, and created a child process using fork(). Thus, I successfully implemented the main shell loop, user input handling, exit condition, and process creation.

make

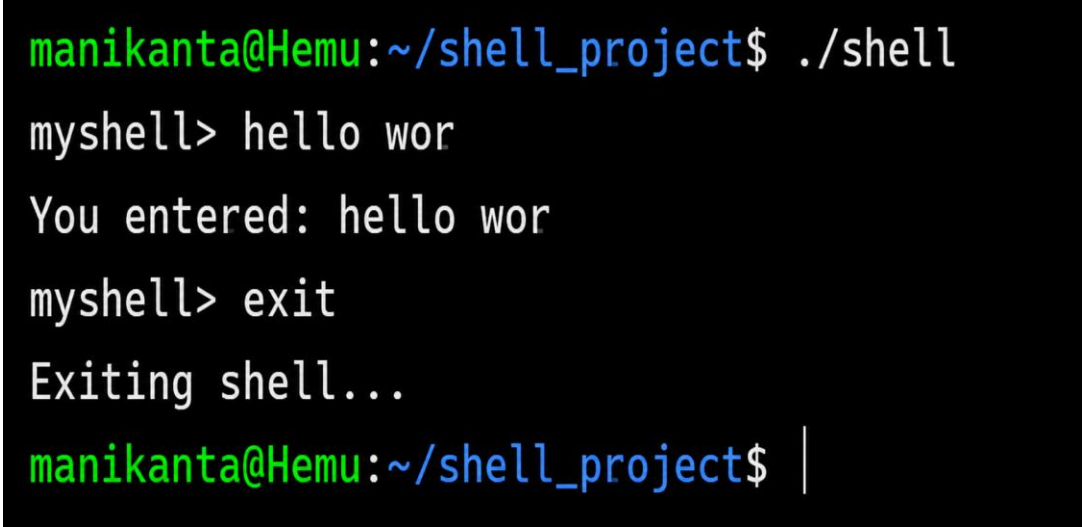
```

manikanta@Hemu:~/shell_project$ make
gcc shell.c -o shell
manikanta@Hemu:~/shell_project$ ./shell
myshell> exit
Exiting shell...
manikanta@Hemu:~/shell_project$ |

```

Report:

I compiled the shell program using the make command and successfully executed it. I tested the exit command and observed that the shell displayed the exit message and terminated correctly. Thus, I successfully built and tested the shell program and verified its exit functionality.

A terminal window with a black background and green and blue text. The prompt is 'manikanta@Hemu:~/shell_project\$'. The user enters './shell', and the prompt changes to 'myshell>'. The user enters 'hello wor', and the shell responds 'You entered: hello wor'. The user enters 'exit', and the shell responds 'Exiting shell...'. The prompt returns to 'manikanta@Hemu:~/shell_project\$'. A vertical line is visible after the prompt.

```
manikanta@Hemu:~/shell_project$ ./shell
myshell> hello wor
You entered: hello wor
myshell> exit
Exiting shell...
manikanta@Hemu:~/shell_project$ |
```

backspace

Report: I executed the shell program and tested it with a multi-character input, hello wor, and the exit command. I observed that the program correctly displayed the entered input and terminated successfully when exit was given. Thus, I successfully tested multi-character command input and the exit functionality of the shell.

fork()

```
manikanta@Hemu:~/shell_project$ make
gcc shell.c -o shell
manikanta@Hemu:~/shell_project$ ./shell
myshell> pwd
Parent waiting for child...
Child process executing: pwd
/home/manikanta/shell_project
Child process completed.
myshell> ls
Parent waiting for child...
Child process executing: ls
Makefile  shell  shell.c
Child process completed.
myshell> exit
Exiting shell...
manikanta@Hemu:~/shell_project$ |
```

Report: I compiled and executed the shell program and tested it with the pwd and ls commands. I observed that the shell successfully created child processes, executed the commands, and waited for the child processes to complete before accepting the next command. Thus, I successfully implemented and tested command execution using fork(), exec(), and wait().

